

7.1 Contexte

Soit la fonction suivante :

```
1 def fonction(x,y = []) :
2     if x == [] :
3         return y
4     else :
5         z = x.pop(0)
6         if z not in y :
7             y.append(z)
8         return fonction(x, y)
```

Problèmes :

- à quoi sert cette fonction ?
- quels sont les paramètres à donner ? leur type ?
- que renvoie la fonction ?

Lors de l'écriture d'une fonction, on peut être amené à l'utiliser, le jour même, le lendemain, une semaine plus tard, un mois plus tard... On peut être amené à la partager lorsqu'on travaille sur un projet. Il est donc essentiel de les documenter, de les commenter.

Présentons donc les choses ainsi.

```
1 def supprimer_doublons_liste(L,y = []) :
2     """
3     Fonction permettant de renvoyer, à partir d'une liste d'entiers, une liste
4     privée des doublons.
5     La liste initiale sera vidée.
6     """
7     if L == [] :
8         return y
9     else :
10        z = L.pop(0)
11        if z not in y :
12            y.append(z)
13        return fonction(L, y)
```

En modifiant le nom de la fonction et en ajoutant une court commentaire, il devient plus simple de partager la fonction ou de la réutiliser plus tard.

Problèmes (encore des problèmes) :

- on veut une fonction qui supprime les doublons d'une liste d'entiers... comment est-on sûrs que la fonction atteint bien cet objectif ?

Pour prouver qu'un algorithme on peut envisager deux méthodes :

- une méthode empirique (expérimentale) se basant sur des tests, sur le contrôle, des données d'entrées. Cette méthode à l'avantage d'être relativement simple à mettre en œuvre. Elle a l'inconvénient de valider le comportement d'une fonction sur des cas « ponctuels ». On ne montre donc pas qu'elle fonctionne dans le cas général.

7.1	Contexte	1
7.2	Spécification d'une fonction	2
7.3	Assertion	3
7.4	Génération de tests unitaires	4

- Spécifications des données attendues en entrée et fournies en sortie/retour.
- Annotation d'un bloc d'instructions par une précondition, une postcondition, une propriété invariante
- Assertion.
- Jeu de tests associé à un programme.

- une méthode formelle, dans le cadre de laquelle on va prouver mathématiquement qu'une fonction atteint bien l'objectif fixé. On parle alors de terminaison et de correction de l'algorithme.

La première méthode fait l'objet du présent chapitre. La seconde fera l'objet d'un chapitre ultérieur.

7.2 Spécification d'une fonction

7.2.1 Spécification à l'aide de commentaires

Définition – Signature

La **signature d'une fonction** définit les entrées et les sorties d'une fonction.

La signature peut comporter par exemple le type des paramètres d'entrées ou de sorties, des conditions sur ces paramètres.

La façon la plus simple de signer une fonction, est d'indiquer des commentaires juste après la déclaration de la fonction.

Exemple –

```

1 def supprimer_doublons_liste(L, y = []) :
2     """
3     Fonction permettant de supprimer les doublons d'une liste.
4     Entrée :
5     * L: list(int) : liste d'entiers.
6     Sortie :
7     * list(int) : liste d'entiers ne comportant aucun doublons.
8     """
9     if L == [] :
10        return y
11    else :
12        z = L.pop(0)
13        if z not in y :
14            y.append(z)
15    return fonction(L, y)

```

Lorsque les commentaires d'une fonction sont déclarés ainsi après la déclaration d'une fonction (triples "), Python génère automatiquement une documentation en utilisant l'instruction `help(supprimer_doublons_liste)`. On parle de docstring.

7.2.2 Annotation d'une fonction– Limite du programme

Depuis la version 3.5 de Python il est possible d'annoter les fonctions.

Définition – Annotations de type

Les annotations de type permettent de donner une information sur le type attendu d'une fonction. Ce ne sont que des indications pour le programmeur. Rien ne l'empêche de ne pas les respecter.

Exemple –

```

1 def addition(a:int, b:int) -> int
  :
2   return a+b

```

```

1 >>> addition(1,1)
2       2
3 >>> addition('a','a')
4       aa

```

Comme précisé, les annotations ne sont que des indications. Mais il pourrait être intéressant de pouvoir s'assurer que l'utilisateur de la fonction respecte les conditions d'utilisation de la fonction que ce soit le type des arguments, ou d'autres conditions.

Remarque

En Python, il est possible d'utiliser `mypy` qui permet de vérifier que lors de l'appel d'une fonction, le typage précisé dans les annotations est bien respecté.

7.3 Assertion

7.3.1 Validation des entrées

Comme suite au paragraphe précédent, en phase de debuggage (ou développement), par opposition à la phase de production, Python prévoit la possibilité de vérifier que les arguments donnés à une fonction respectent bien les préconisations du concepteur.

En utilisant l'instruction `assert`, le programme s'interrompt quand une erreur est détectée.

Exemple –

Dans la lignée des annotations de type, on peut tester le type d'argument donné à une fonction.

```

1 def addition(a:int, b:int) -> int
  :
2   assert type(a) == int and
  type(b) == int
3   return a+b

```

```

1 >>> addition(1,1)
2       2
3 >>> addition('a','a')
4       (...)
5       AssertionError

```

Si les arguments sont du mauvais type, Python déclenche une `AssertionError`.

Exemple –

Prenons un type d'exemple que nous rencontrerons plus souvent : l'algorithme fonctionne seulement si des préconditions sont remplies.

Par exemple dans le cas de la résolution de l'équation $f(x) = 0$, il est nécessaire que f soit continue, monotone et que $f(a)$ et $f(b)$ soient de signes différents. Il faut aussi que ε soit positif. On peut donc procéder de la façon suivante.

```

1 def dichotomie(f, a:float , b:float , epsilon:float) -> float :
2   """Zéro de f sur [a,b] à epsilon près, par dichotomie
3   Préconditions : f(a) * f(b) <= 0
4   f continue sur [a,b]
5   epsilon > 0"""
6   assert (f(a) * f(b) <= 0) and (epsilon > 0)
7   c, d = a, b

```

```

8     fc, fd = f(c), f(d)
9     while d - c > 2 * epsilon:
10         m = (c + d) / 2.
11         fm = f(m)
12         if fc * fm <= 0:
13             d, fd = m, fm
14         else:
15             c, fc = m, fm
16     return (c + d) / 2.

```

7.3.2 Vers la gestion d'exceptions – hors programme

L'avantage des assertions et leur facilité à être mise en œuvre. L'inconvénient est qu'il n'est pas possible de savoir précisément ce qui a généré l'erreur d'assertion : un problème de précondition? de type? autre?

Du fait de leur manque de précision, il peut être difficile de les gérer.

Un premier pas vers la gestion d'erreur est de donner à l'utilisateur la raison pour laquelle la fonction génère une erreur. Pour cela, on va *lever une exception*.

```

1 def dichotomie(f, a:float , b:float , epsilon:float) -> float :
2     """Zéro de f sur [a,b] à epsilon près, par dichotomie
3     Préconditions : f(a) * f(b) <= 0
4     f continue sur [a,b]
5     epsilon > 0"""
6     if f(a) * f(b) > 0 :
7         raise Exception("f(a) et f(b) sont de même signe")
8     elif epsilon < 0 :
9         raise Exception("epsilon est négatif")
10
11     c, d = a, b
12     fc, fd = f(c), f(d)
13     while d - c > 2 * epsilon:
14         m = (c + d) / 2.
15         fm = f(m)
16         if fc * fm <= 0:
17             d, fd = m, fm
18         else:
19             c, fc = m, fm
20     return (c + d) / 2.

```

```

1 >>> dichotomie(f, 0, 1, 0.00001)
2 (...)
3 Exception: f(a) et f(b) sont de même signe

```

7.4 Génération de tests unitaires

On a vu comment vérifier que les données d'entrées d'une fonction sont compatibles avec ce qu'attendait le concepteur de la fonction. On va maintenant chercher, grâce à des tests unitaires, si le résultat attendu est bien celui déterminé par une fonction.

Exemple –

<pre> 1 def addition(a:int, b:int) -> int : 2 assert type(a) == int and type(b) 3 == int 3 return a+b </pre>	<pre> 1 >>> addition(1,1) == 2 2 True 3 >>> addition(1,1) == 3 4 False </pre>
---	---

Ces tests ont l'avantage d'être simples à mettre en œuvre mais ne garantissent pas que la fonction « marche » dans tous les cas.

Il est alors possible de créer une fonction de tests.

```

1 def test_addition() :
2     assert addition(1,1) == 2
3     assert addition(1,1) !=3

```

Des jeux de tests pertinents peuvent être des tests sur les cas limites. Dans le cas de la recherche dichotomique, on peut se demander si la boucle `while` n'est pas réalisée une fois de trop par rapport à la valeur de `epsilon`.

```

1 >>> res = dichotomie(f, 0, 1, epsilon)
2 >>> f(res) < epsilon # MAUVAIS test : f(res) peut être négatif
3         True
4 >>> abs(f(res)) < epsilon
5         False
6 >>> # L'algorithme dichotomique doit itérer une fois de plus ?

```

En modifiant l'algorithme, et en itérant une fois de plus, le test reste faux. La condition d'arrêt de l'algorithme ne permet pas de trouver x tel que $f(x) < \varepsilon$ mais de trouver une valeur de x telle que l'écart entre deux solutions consécutives est inférieur à ε . Il faut donc clarifier l'objectif de la fonction pour proposer un test plus adapté.